Projeto 6 – DTMF

Camada Física da Computação · Engenharia da Computação · Documentação do código

1. O que a atividade pede

O projeto pede um sistema **DTMF** (*Dual Tone Multi Frequency*) feito em software. DTMF é o sinal que o telefone emite quando apertamos uma tecla: cada tecla gera **duas senoides somadas**, uma de frequência baixa (da linha) e outra de frequência alta (da coluna) do teclado.

	1209 Hz	**1336 Hz**	**1477 Hz**	**1633 Hz**
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	7	8	9	C
941 Hz	X	0	#	D

O sistema tem duas partes, que rodam em computadores diferentes (ou um computador e um celular):

Lado emissor (encoder)

- Perguntar ao usuário qual número, de 0 a 9, ele quer digitar.
- Emitir por alguns segundos as duas frequências da tecla, somadas.
- Plotar o sinal no domínio do tempo.
- Plotar o sinal no domínio da frequência (transformada de Fourier).
- Opcional: salvar o sinal gerado em arquivo.

Lado receptor (decoder)

- Gravar o som pelo microfone, já com o emissor tocando (sem gravar silêncio).
- Calcular a transformada de Fourier do sinal gravado.
- Identificar os picos: **no mínimo 5**.
- Descobrir e imprimir a tecla a partir dos picos.
- Plotar o sinal recebido no tempo e a sua transformada de Fourier.

Critérios de avaliação

1. Produzir o sinal corretamente (mesmo sem reconhecer a tecla).
2. Reconhecer no receptor a tecla digitada em **todas** as tentativas.
3. Reconhecer a tecla, plotar os gráficos corretamente e **saber interpretar os gráficos**.

> **Sobre a tabela:** a tabela do enunciado mostra 679 Hz e 825 Hz, mas o padrão DTMF usa **697 Hz** e **852 Hz**. O próprio exemplo de saída do enunciado usa 852 Hz para a tecla 7, então o código usa os valores do padrão. Para usar os valores do PDF, basta trocar a lista `LINHAS` em `suaBibSignal.py`.

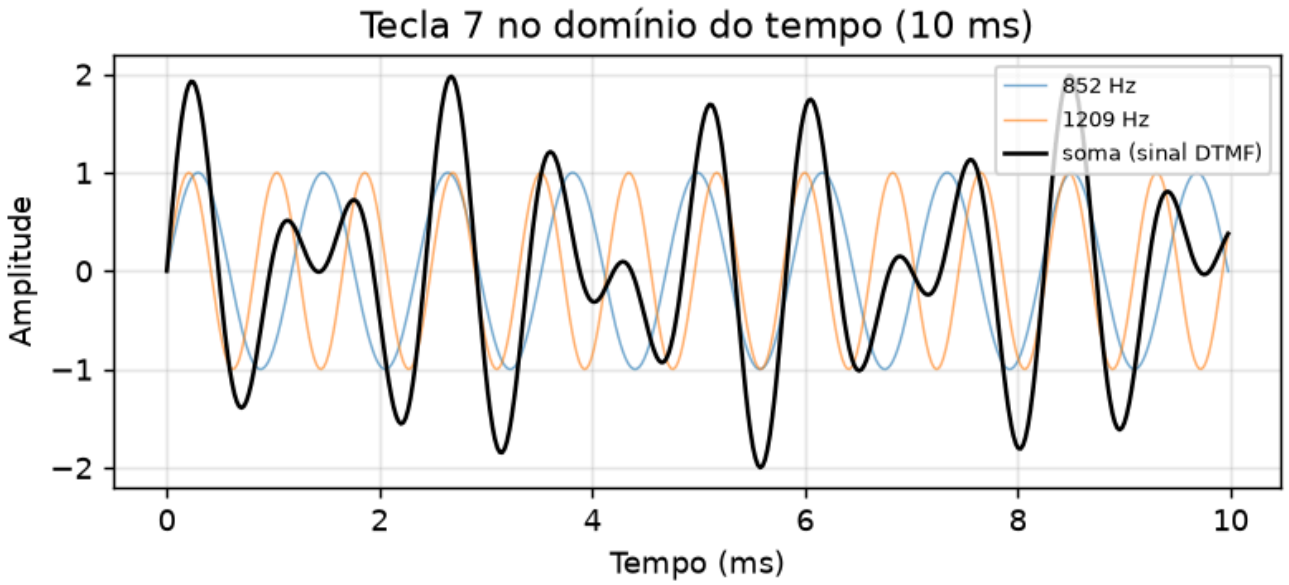
2. Conceitos usados

Amostragem

Um som digital é uma lista de números: o valor da onda medido a intervalos regulares. A **taxa de amostragem** usada é **44 100 amostras por segundo**, a das placas de som. Pelo teorema de Nyquist, ela consegue representar frequências até a metade disso, 22 050 Hz, bem acima da maior frequência DTMF (1633 Hz).

Soma de senoides

Cada senoide é gerada com `A·sin(2π·f·t)`. O sinal da tecla é a soma de duas delas. No tempo, a soma parece uma onda "irregular" que sobe e desce de amplitude (efeito de *batimento*), porque as duas ondas ficam alternadamente em fase e fora de fase.



As duas senoides da tecla 7 e a soma delas (em preto).

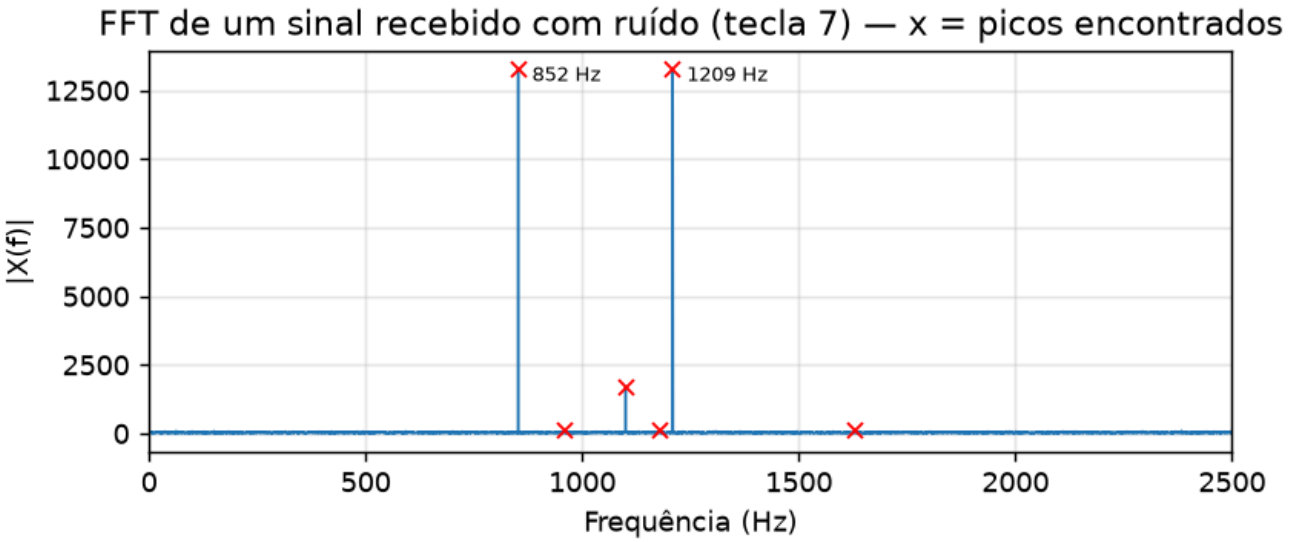
Transformada de Fourier (FFT)

No tempo é difícil saber quais frequências estão "misturadas" no sinal. A transformada de Fourier passa o sinal para o **domínio da frequência**: o resultado mostra quanto de cada frequência existe no sinal. Um sinal DTMF limpo vira **dois picos**, exatamente nas frequências da tecla. Um sinal gravado pelo microfone tem também ruído, que aparece como pequenos picos espalhados.

A **resolução** da FFT (a distância entre duas frequências do eixo x) é Δf = 1 / duração da gravação. Com 3 s, Δf ≈ 0,33 Hz, bem mais fina que a distância entre as frequências DTMF (no mínimo 73 Hz).

Detecção de picos

Um **pico** é um ponto do espectro maior que os vizinhos. O decoder acha todos os picos entre 500 e 2000 Hz, fica com os 6 maiores e procura na tabela qual frequência de linha e qual de coluna têm os picos mais fortes por perto. Essa linha e essa coluna dão a tecla.



Exemplo: o maior pico perto de uma linha está em 852 Hz e o maior perto de uma coluna em 1209 Hz, logo a tecla é 7. Os demais "x" são picos de ruído.

3. Organização e funcionamento

```
┌──────────── encode.py (emissor) ────────────┐          ┌──────────── decode.py (receptor) ────────────┐
│ usuário digita a tecla                      │          │ grava 3 s do microfone (sd.rec)              │
│ DTMF[tecla] → (f_linha, f_coluna)           │          │ FFT (calcFFT) → espectro |X(f)|              │
│ gera sin(f_linha) + sin(f_coluna), 5 s      │   som    │ find_peaks em 500–2000 Hz → 6 maiores picos  │
│ normaliza e toca (sd.play)          ────────┼──────────► linha e coluna mais fortes → TECLA           │
│ plota tempo + FFT, salva .wav               │   (ar)   │ plota tempo + FFT com os picos               │
└─────────────────────────────────────────────┘          └──────────────────────────────────────────────┘
                ▲                                                          ▲
                └──────────── suaBibSignal.py: tabela DTMF, generateSin, calcFFT, plotFFT ────────────┘
```

Arquivos:

- `suaBibSignal.py`: biblioteca de apoio, usada pelos dois programas (tabela DTMF, geração de senoide, FFT).
- `encode.py`: o emissor.
- `decode.py`: o receptor.

Como executar

```
pip install numpy scipy matplotlib sounddevice

python encode.py            # no computador emissor: digite a tecla
python decode.py            # no receptor: rode com o som já tocando
python decode.py tecla_7.wav  # teste sem microfone, usando o .wav salvo pelo encoder
```

Saída típica do receptor:

```
freq de pico sao 852.00 Hz
freq de pico sao 1209.00 Hz
...  (mais 4 picos de ruído)
Frequências identificadas: 852 Hz + 1209 Hz
TECLA: 7
```

4. Explicação linha a linha

Cada bloco mostra as linhas do arquivo (com a numeração real) e, logo abaixo, o que elas fazem. Linhas em branco foram omitidas.

4.1 suaBibSignal.py — biblioteca de apoio

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.fftpack import fft
4  from scipy import signal as window
```

Linhas 1–4 Importa as bibliotecas. **numpy** faz as contas com vetores, **matplotlib** desenha os gráficos, **fft** (do scipy) calcula a Transformada Rápida de Fourier e **scipy.signal** fornece a janela de Hann. Ele é importado com o apelido `window` para não confundir com a variável `signal` usada no resto do código.

```
7  # Tabela DTMF (frequências padrão ITU)
8  LINHAS = [697, 770, 852, 941]
9  COLUNAS = [1209, 1336, 1477, 1633]
```

Linhas 7–9 As frequências DTMF padrão. `LINHAS` são as frequências baixas (uma por linha do teclado) e `COLUNAS` são as frequências altas (uma por coluna). A tabela do PDF mostra 679 e 825 Hz, mas os valores do padrão são 697 e 852 Hz, que é o que o próprio exemplo de saída do enunciado usa (852 Hz para a tecla 7).

```
10 TECLAS = [
11     ["1", "2", "3", "A"],
12     ["4", "5", "6", "B"],
13     ["7", "8", "9", "C"],
14     ["X", "0", "#", "D"],
15 ]
```

Linhas 10–15 O teclado como matriz 4×4. A tecla na linha *i* e coluna *j* é `TECLAS[i][j]`.

```
17 DTMF = {}
18 for i, f_linha in enumerate(LINHAS):
19     for j, f_coluna in enumerate(COLUNAS):
20         DTMF[TECLAS[i][j]] = (f_linha, f_coluna)
```

Linhas 17–20 Monta o dicionário `DTMF`, que associa cada tecla ao par de frequências dela. Os dois `for` com `enumerate` percorrem todas as combinações linha×coluna. Exemplo: `DTMF['7'] = (852, 1209)`. O encoder usa esse dicionário para converter tecla em frequências.

```
23 class signalMeu:
24
25     def __init__(self):
26         self.init = 0
```

Linhas 23–26 A classe `signalMeu`, a "biblioteca de apoio" citada nos arquivos-modelo. O `__init__` não faz nada de importante: a classe só agrupa as funções de sinal.

```
28      def generateSin(self, freq, amplitude, time, fs):
29          n = int(time * fs)
30          x = np.linspace(0.0, time, n, endpoint=False)
31          s = amplitude * np.sin(2 * np.pi * freq * x)
32          return (x, s)
```

Linhas 28–32 **generateSin** gera uma senoide amostrada. `n = time·fs` é o número de amostras (5 s × 44100 = 220 500). `linspace(..., endpoint=False)` cria os instantes 0, 1/fs, 2/fs, … (um instante por amostra). Depois aplica a fórmula `A·sin(2πft)` pedida no enunciado. Devolve o vetor de tempo e o vetor de amplitudes.

```
34      def calcFFT(self, signal, fs):
35          # https://docs.scipy.org/doc/scipy/reference/tutorial/fftpack.html
36          N = len(signal)
37          W = window.windows.hann(N)
38          T = 1 / fs
39          xf = np.linspace(0.0, 1.0 / (2.0 * T), N // 2)
40          yf = fft(signal * W)
41          return (xf, np.abs(yf[0:N // 2]))
```

Linhas 34–41 **calcFFT** calcula o espectro (Fourier) do sinal. Passo a passo:
• `N` é o número de amostras.
• `W` é a janela de Hann. Ela multiplica o sinal e suaviza o começo e o fim do trecho. Sem ela, o corte abrupto da gravação espalha energia por várias frequências (vazamento espectral) e os picos ficam "gordos".
• `xf` é o eixo de frequências, de 0 até fs/2 = 22 050 Hz (a frequência de Nyquist, a maior que dá para representar).
• `yf = fft(...)` é a transformada. O resultado é complexo e simétrico, então só a primeira metade (frequências positivas) é guardada, e `np.abs` pega o módulo, que é a "quantidade" de cada frequência.

```
43      def plotFFT(self, signal, fs, titulo="Fourier"):
44          x, y = self.calcFFT(signal, fs)
45          plt.figure()
46          plt.plot(x, np.abs(y))
47          plt.title(titulo)
48          plt.xlabel("Frequência (Hz)")
49          plt.ylabel("|X(f)|")
50          plt.grid(True)
```

Linhas 43–50 **plotFFT** chama `calcFFT` e desenha |X(f)| × frequência numa figura nova, com título e eixos.

4.2 encode.py — emissor

```
1  #importe as bibliotecas
2  from suaBibSignal import *
3  import numpy as np
4  import sounddevice as sd
5  import matplotlib.pyplot as plt
6  from scipy.io import wavfile
7  import signal as sig
8  import sys
```

Linhas 1–8 Importações. `from suaBibSignal import *` traz a classe `signalMeu` e a tabela `DTMF`. **sounddevice** toca o som na placa de áudio. **wavfile** salva o .wav. **signal** (do Python) e **sys** servem para tratar o Ctrl+C.

```
10 FS = 44100        # taxa de amostragem
11 DURACAO = 5       # segundos de som emitido
12 AMPLITUDE = 1
```

Linhas 10–12 Constantes: taxa de amostragem de 44 100 Hz (a das placas de som, como pede o enunciado), 5 s de som e amplitude 1.

```
15 #funções caso queriram usar para sair...
16 def signal_handler(signal, frame):
17         print('You pressed Ctrl+C!')
18         sys.exit(0)
```

Linhas 15–18 Função chamada quando o usuário aperta Ctrl+C: imprime uma mensagem e encerra o programa.

```
20 #converte intensidade em Db, caso queiram ...
21 def todB(s):
22     sdB = 10*np.log10(s)
23     return(sdB)
```

Linhas 20–23 Converte intensidade para decibéis (10·log10). Veio do arquivo-modelo e não é usada.

```
26 def main():
27     sig.signal(sig.SIGINT, signal_handler)
28     signal = signalMeu()
```

Linhas 26–28 Início do `main`. Registra o tratador de Ctrl+C e cria o objeto da biblioteca de apoio.

```
30     print("Inicializando encoder")
31     print("Aguardando usuário")
32     NUM = input("Digite uma tecla (0-9, A-D, X ou #): ").strip().upper()
33     while NUM not in DTMF:
34         NUM = input("Tecla inválida. Digite novamente: ").strip().upper()
```

Linhas 30–34 Pede a tecla ao usuário. `strip()` tira espaços e `upper()` aceita "a" como "A". O `while` repete a pergunta enquanto a tecla não estiver na tabela DTMF.

```
36      f_linha, f_coluna = DTMF[NUM]
```

Linha 36 Busca no dicionário as duas frequências da tecla: `f_linha` (baixa) e `f_coluna` (alta).

```
38      print("Gerando Tons base")
39      t, s1 = signal.generateSin(f_linha, AMPLITUDE, DURACAO, FS)
40      _, s2 = signal.generateSin(f_coluna, AMPLITUDE, DURACAO, FS)
41      tone = s1 + s2
```

Linhas 38–41 Gera as duas senoides, cada uma com amplitude 1, 5 s e 44 100 amostras/s, e **soma** as duas. Essa soma é o sinal DTMF. Para a tecla 7: sin(2π·852·t) + sin(2π·1209·t).

```
43      # a soma pode chegar a 2; normaliza para [-1, 1] para não saturar a placa de som
44      tone_play = tone / np.max(np.abs(tone))
```

Linhas 43–44 A soma de duas senoides de amplitude 1 pode chegar a ±2, mas a placa de som só aceita valores entre −1 e +1. Acima disso o som satura (fica distorcido e cria frequências falsas). Por isso o sinal é dividido pelo seu valor máximo antes de tocar. Os gráficos continuam usando o sinal original, com amplitude até 2.

```
46      # opcional: salva o sinal gerado em arquivo
47      wavfile.write("tecla_{}.wav".format(NUM if NUM != "#" else "hash"), FS,
48                    (tone_play * 32767).astype(np.int16))
```

Linhas 46–48 Parte opcional do enunciado: salva o sinal num .wav. O arquivo WAV guarda inteiros de 16 bits (−32768 a 32767), então o sinal é multiplicado por 32767 e convertido para int16. É a etapa de **quantização/codificação** da aula de digitalização. O "#" vira "hash" no nome porque não é um caractere bom para nome de arquivo.

```
50      print("Gerando Tom referente ao símbolo : {}".format(NUM))
51      print("Frequências : {} + {}".format(f_linha, f_coluna))
52      print("Executando as senoides (emitindo o som)")
53      sd.play(tone_play, FS)
54      # aguarda fim do audio
55      sd.wait()
```

Linhas 50–55 Imprime as mensagens de status e toca o som. `sd.play` começa a tocar sem bloquear o programa, e `sd.wait` espera o som terminar.

```
57      print("Plotando")
58      # sinal no tempo: apenas os primeiros 10 ms (alguns períodos)
59      n = int(0.01 * FS)
60      plt.figure()
61      plt.plot(t[:n] * 1000, tone[:n])
62      plt.title("Sinal emitido - tecla {} ({} Hz + {} Hz)".format(NUM, f_linha, f_coluna))
63      plt.xlabel("Tempo (ms)")
64      plt.ylabel("Amplitude")
65      plt.grid(True)
```

Linhas 57–65 Gráfico no domínio do tempo. Com 44 100 amostras por segundo, plotar os 5 s inteiros viraria só uma mancha, então só os primeiros 10 ms (441 amostras) são desenhados. Nesse trecho dá para ver as ondas. O eixo x é multiplicado por 1000 para ficar em ms.

```
67      signal.plotFFT(tone, FS, "Fourier do sinal emitido - tecla {}".format(NUM))
68      plt.xlim(0, 2500)
69      # Exibe gráficos
70      plt.show()
```

Linhas 67–70 Gráfico da FFT do sinal emitido, mostrado até 2500 Hz, onde estão as frequências DTMF. Devem aparecer exatamente dois picos, nas duas frequências da tecla. `plt.show()` abre as janelas dos gráficos.

```
73  if __name__ == "__main__":
74      main()
```

Linhas 73–74 Executa o `main` só quando o arquivo é rodado diretamente (`python encode.py`), e não quando ele é importado.

4.3 decode.py — receptor

```
1  #Importe todas as bibliotecas
2  from suaBibSignal import *
3  import numpy as np
4  import sounddevice as sd
5  import matplotlib.pyplot as plt
6  from scipy.signal import find_peaks
7  from scipy.io import wavfile
8  import time
9  import sys
```

Linhas 1–9 Importações. A diferença em relação ao encoder é **find_peaks** (do scipy), a função que acha os picos (máximos locais) no espectro. O enunciado pede "funções para identificar picos"; ela faz o papel do `peakutils` citado no arquivo-modelo.

```
11 FS = 44100
12 DURACAO = 3            # segundos de gravação
13 NUM_PICOS = 6          # o enunciado pede no mínimo 5
14 DIST_MIN_HZ = 5        # picos mais próximos que isso são considerados o mesmo
15 TOLERANCIA_HZ = 20     # distância máxima entre um pico e a frequência da tabela
```

Linhas 11–15 Parâmetros de ajuste do detector:

- `DURACAO = 3`: grava 3 s. Gravar mais tempo melhora a resolução da FFT (Δf = 1/3 s ≈ 0,33 Hz).
- `NUM_PICOS = 6`: quantos picos imprimir (o enunciado pede no mínimo 5).
- `DIST_MIN_HZ = 5`: picos a menos de 5 Hz um do outro contam como um só (sugestão do arquivo-modelo).
- `TOLERANCIA_HZ = 20`: um pico a até 20 Hz de uma frequência da tabela ainda "casa" com ela, porque o som real pode chegar levemente deslocado.

```
18 #funcao para transformas intensidade acustica em dB, caso queira usar
19 def todB(s):
20     sdB = 10*np.log10(s)
21     return(sdB)
```

Linhas 18–21 Conversão para dB, vinda do arquivo-modelo. Não é usada.

```
24  def gravar():
25      sd.default.samplerate = FS
26      sd.default.channels = 1
27      numAmostras = DURACAO * FS
28
29      print("A captação começará em 2 segundos")
30      time.sleep(2)
31      print("Gravação inicializada")
32      audio = sd.rec(int(numAmostras), FS, channels=1)
33      sd.wait()
34      print("...     FIM")
35      return audio[:, 0]
```

Linhas 24–35 **gravar** grava o áudio do microfone.
• Configura o sounddevice com 44 100 Hz e 1 canal (mono basta: não precisamos de estéreo).
• `numAmostras = duração × taxa` = 3 × 44 100 = 132 300 amostras.
• Espera 2 s antes de gravar, para dar tempo de o outro computador já estar tocando. O enunciado pede para não gravar silêncio.
• `sd.rec` inicia a gravação e `sd.wait` espera ela terminar.
• `sd.rec` devolve uma matriz (amostras × canais). `audio[:, 0]` pega a coluna do canal 0 e transforma em lista simples, como pede o comentário do modelo (“extraia a parte que interessa”).

```
38  def ler_wav(caminho):
39      fs, dados = wavfile.read(caminho)
40      if dados.ndim > 1:
41          dados = dados[:, 0]
42      if dados.dtype == np.int16:
43          dados = dados / 32768.0
44      return fs, dados.astype(float)
```

Linhas 38–44 **ler_wav** é um extra para testar sem microfone: lê um .wav (por exemplo, o salvo pelo encoder). Se o arquivo for estéreo, pega um canal. Se estiver em int16, divide por 32768 para voltar à escala −1…+1.

```
47  def encontrar_picos(xf, yf):
48      df = xf[1] - xf[0]
49      # só a faixa de interesse do DTMF
50      faixa = (xf > 500) & (xf < 2000)
51      idx_faixa = np.where(faixa)[0]
52      picos, _ = find_peaks(yf[faixa], distance=max(1, int(DIST_MIN_HZ / df)))
53      picos = idx_faixa[picos]
54      # mantém os NUM_PICOS mais altos, ordenados por frequência
55      picos = picos[np.argsort(yf[picos])[::-1][:NUM_PICOS]]
56      return np.sort(picos)
```

Linhas 47–56 **encontrar_picos** é o coração do detector.
• `df` é o espaçamento entre duas frequências consecutivas da FFT (resolução em Hz).
• `faixa` limita a busca a 500–2000 Hz, onde estão todas as frequências DTMF. Fora disso só há ruído e zumbido da rede elétrica (60 Hz).
• `find_peaks(..., distance=...)` acha todos os máximos locais, exigindo uma distância mínima de 5 Hz (convertida para número de pontos: 5/df).
• `idx_faixa[picos]` converte os índices do recorte de volta para índices do vetor completo.
• `argsort(...)[::-1][:NUM_PICOS]` ordena os picos pela altura, do maior para o menor, e fica com os 6 maiores.
• Por fim, ordena por frequência, para imprimir em ordem crescente.

```
59  def identificar_tecla(xf, yf, picos):
60      def melhor(freqs_tabela):
61          # para cada frequência da tabela, a amplitude do maior pico dentro da tolerância
62          scores = []
63          for f in freqs_tabela:
64              perto = [p for p in picos if abs(xf[p] - f) <= TOLERANCIA_HZ]
65              scores.append(max((yf[p] for p in perto), default=0))
66          i = int(np.argmax(scores))
67          return i, scores[i]
68
69      i, s_linha = melhor(LINHAS)
70      j, s_coluna = melhor(COLUNAS)
71      if s_linha == 0 or s_coluna == 0:
72          return None, None, None
73      return TECLAS[i][j], LINHAS[i], COLUNAS[j]
```

Linhas 59–73 **identificar_tecla** descobre a tecla a partir dos picos.
• A função interna `melhor` recebe uma lista de frequências da tabela (as 4 linhas ou as 4 colunas). Para cada uma, procura entre os picos encontrados os que estão a até 20 Hz dela e guarda a altura do maior (ou 0 se nenhum estiver perto). Depois escolhe a frequência da tabela com o pico mais alto.
• Isso é feito uma vez para as linhas e outra para as colunas. Assim, um pico de ruído perto de 770 Hz não "ganha" do pico verdadeiro em 852 Hz, porque o verdadeiro é bem mais alto.
• Se não houver nenhum pico perto de alguma linha ou de alguma coluna, devolve `None` (nenhuma tecla). Senão, devolve `TECLAS[i][j]` e as duas frequências.

```
76  def main():
77      signal = signalMeu()
78
79      if len(sys.argv) > 1:
80          fs, dados = ler_wav(sys.argv[1])
81          print("Lendo arquivo {}".format(sys.argv[1]))
82      else:
83          fs, dados = FS, gravar()
```

Linhas 76–83 Início do `main`. Se o programa foi chamado com um argumento (`python decode.py arquivo.wav`), lê o arquivo; senão, grava pelo microfone.

```
85      t = np.linspace(0, len(dados) / fs, len(dados), endpoint=False)
```

Linha 85 Cria o vetor de tempo com `linspace`: um instante para cada amostra, como pede o modelo.

```
87      # plot do sinal recebido no tempo (apenas 20 ms para enxergar as ondas)
88      n = int(0.02 * fs)
89      meio = len(dados) // 2
90      plt.figure()
91      plt.plot(t[meio:meio + n], dados[meio:meio + n])
92      plt.title("Sinal recebido (trecho de 20 ms)")
93      plt.xlabel("Tempo (s)")
94      plt.ylabel("Amplitude")
95      plt.grid(True)
```

Linhas 87–95 Gráfico do sinal recebido no tempo. Mostra só 20 ms tirados do **meio** da gravação, onde o som já está estável (no começo pode haver um transiente do microfone).

```
97      xf, yf = signal.calcFFT(dados, fs)
98      picos = encontrar_picos(xf, yf)
99
100     for p in picos:
101         print("freq de pico sao {:.2f} Hz".format(xf[p]))
```

Linhas 97–101 Calcula a FFT do sinal gravado, acha os picos e imprime a frequência de cada um.

```
103     tecla, f_linha, f_coluna = identificar_tecla(xf, yf, picos)
104     if tecla is None:
105         print("Nenhuma tecla DTMF identificada")
106     else:
107         print("Frequências identificadas: {} Hz + {} Hz".format(f_linha, f_coluna))
108         print("TECLA: {}".format(tecla))
```

Linhas 103–108 Identifica a tecla e imprime o resultado.

```
110     plt.figure()
111     plt.plot(xf, yf)
112     plt.plot(xf[picos], yf[picos], "rx")
113     plt.title("Fourier do sinal recebido" + ("" if tecla is None else " - tecla {}".format(tecla)))
114     plt.xlabel("Frequência (Hz)")
115     plt.ylabel("|X(f)|")
116     plt.xlim(0, 2500)
117     plt.grid(True)
118
119     ## Exiba gráficos do fourier do som gravados
120     plt.show()
```

> **Linhas 110–120** Gráfico da FFT do sinal recebido, com um "x" vermelho em cada pico encontrado, e abre as janelas.

```
123 if __name__ == "__main__":
124     main()
```

> **Linhas 123–124** Mesmo padrão do encoder: o `main` só roda quando o arquivo é executado diretamente.

5. Como interpretar os gráficos

- **Tempo (emissor):** uma onda que varia de amplitude, entre −2 e +2. Não é uma senoide pura porque é a soma de duas frequências.
- **Fourier (emissor):** exatamente dois picos, nas duas frequências da tecla, e praticamente nada no resto. É o sinal ideal.
- **Tempo (receptor):** a mesma forma, mas com amplitude menor (o som perde energia no ar) e com ruído por cima.
- **Fourier (receptor):** os dois picos principais continuam nas mesmas frequências e são os mais altos. Os outros picos marcados são ruído do ambiente, zumbido elétrico ou distorção do alto-falante e do microfone. Por isso o decoder não pega simplesmente "os dois maiores picos": ele compara cada pico com a tabela.
- **Por que mostrar só alguns milissegundos no tempo?** Com 44 100 amostras por segundo, a 1209 Hz há cerca de 36 amostras por período. Em segundos inteiros o gráfico vira uma mancha. Em 10–20 ms dá para ver as ondas.

6. Ajustes caso o reconhecimento falhe

- Aumente o volume ou aproxime o microfone do alto-falante.
- Comece a gravação só depois que o som já estiver tocando.
- Se a frequência chegar deslocada, aumente `TOLERANCIA_HZ` em `decode.py`.
- Se o ambiente for muito ruidoso, aumente `DURACAO` da gravação: a FFT fica mais precisa e o pico verdadeiro se destaca mais.